

Prompt 1:

Read README.md and inspect the Frogger code.

Fix only the broken vehicle collision detection.

The current collision logic incorrectly compares the frog's grid column with the vehicle's grid position. Vehicles move continuously in pixel space.

Change collision detection to use the actual pixel-space rectangles of the frog and vehicles using `pygame.Rect` and `colliderect()`.

Do not add lives, score, timer, or other features yet.

After making the change, show me the files changed and the relevant code.

Prompt 2:

Now implement the 3-lives feature from README.md.

Requirements:

- Start with 3 lives.
- Vehicle collision removes 1 life.
- After collision, visibly show the hit briefly and then respawn the frog at its starting position.
- Display remaining lives.
- At 0 lives, stop the game and display "Game Over".
- Press R to restart with 3 lives.

- Keep the accurate rectangle collision detection.
- Do not implement score or timer yet.

Prompt 3:

Now implement the score and win feature from README.md.

Requirements:

- Detect when the frog reaches the goal row.
- Increase the score.
- Display the score during gameplay.
- Reaching the goal ends the game.
- Display "You Won!".
- Press R to restart the game and reset the score.
- Keep the existing lives and collision systems working.

Prompt 4:

Now implement the 30-second timer feature from README.md.

Requirements:

- Each attempt starts with 30 seconds.
- Display the countdown timer.
- Decrease the timer once per second.
- If the timer reaches 0 before reaching the goal, remove one life.
- If lives remain, respawn the frog and reset the timer to 30 seconds.
- If the final life is lost due to timeout, display "Game Over".
- Reaching the goal before timeout displays "You Won!".

- Press R to completely restart the game with 3 lives, score 0, and timer 30 seconds.
- Keep all existing collision, lives, score, and movement functionality working.